MARS COLONY

Interactive Resource Handler

Operating Systems Project Report

Project Title: Mars Colony Interactive Resource Handler

Algorithm: Banker's Algorithm (Deadlock Avoidance)

Language: C (C11 standard)

Graphics Library: Raylib

Platform: Linux/Unix

Authors: Nimra Mehmood (24K-0807), Shorouq Iqbal (24K-0914)

Course: CS 3012: Operating Systems

Date: May 2026

This project is an interactive, multithreaded visualization of the Banker's Algorithm for operating systems deadlock avoidance. The application is built in C with the Raylib graphics library and features a creative Mars Colony theme. The Banker's Algorithm is a critical concept in operating systems that prevents deadlock by determining whether allocating a resource is safe before granting it to a process. This project demonstrates this complex algorithm through an engaging, visual, and interactive interface.

1.1 Objective

The primary objective of this project is to create an educational tool that visualizes the Banker's Algorithm in action, helping students understand how deadlock avoidance works in multi-process environments. The application demonstrates the algorithm's safety checks, resource allocation, and recovery mechanisms through real-time visualization and concurrent stress testing.

The Mars Colony Interactive Resource Handler is a sophisticated educational simulation that brings the Banker's Algorithm to life. It simulates a Mars colony where multiple processes (represented as missions competing for limited resources) compete for limited assets (O2 Tanks, Batteries, Drones, etc.). The system ensures no deadlock occurs by carefully managing resource allocation using the Banker's Algorithm, which checks if allocation requests are safe before granting them.

2.1 Key Features

Feature	Description
Banker's Algorithm	Implements deadlock avoidance using safety checks
Multi-threaded	Multiple concurrent processes running simultaneously
Visual Simulation	Mars Colony theme with Raylib graphics
Real-time Monitoring	Live display of resource allocation and process states
Stress Testing	Handles concurrent resource requests under load
Interactive UI	Responsive controls for starting, pausing, and analyzing

The Mars Colony system manages resource allocation among multiple processes using the Banker's Algorithm. The core functionalities include:

- Process Management: Create, manage, and track multiple concurrent processes
- Resource Allocation: Safe allocation of multiple resource types (O2 Tanks, Batteries, Drones, etc.)
- Safety Checking: Verify allocation requests against the Banker's Algorithm safety criteria
- Deadlock Prevention: Detect and prevent unsafe allocations that could lead to deadlock
- State Visualization: Real-time display of process states and resource availability
- Concurrent Execution: Handle multiple threads accessing shared resources safely
- Performance Metrics: Track allocation attempts, successes, and failures

The Banker's Algorithm is a classic deadlock avoidance algorithm developed by Edsger Dijkstra. It works on the principle that a resource allocation request is only granted if the resulting system state is safe. A state is considered safe if there exists at least one sequence in which all processes can complete without deadlock.

4.1 Key Concepts

Safe State: A state where all processes can be completed even if they request maximum resources. The algorithm maintains a safety sequence to verify this.

Available Resources: Resources currently available for allocation to processes.

Maximum Need: The maximum resources each process might need.

Allocated Resources: Resources currently held by each process.

Need Matrix: Remaining resources needed by each process (Maximum - Allocated).

4.2 Algorithm Steps

1. Check if requested resources are available
2. Allocate resources tentatively to the requesting process
3. Run safety algorithm to check if state remains safe
4. If safe: confirm allocation and update system state
5. If unsafe: deny allocation and restore previous state

Technology	Purpose	Version/Details
C Language	Core implementation	C11 standard, GCC compiler
Raylib	Graphics and visualization	Lightweight 2D graphics library
Threads (pthreads)	Multi-threading support	POSIX threads f concurrent processes
Mutex (pthread_mutex_t)	Synchronization	Thread-safe resource access sys_lock
Linux/Unix	Operating System	Ubuntu/Debian systems
GCC	Compiler	GNU C Compiler
GNU Make	Build system	Makefile-based compilation

6.1 The arrays needed:

The header defines global 2D arrays and a **SystemState** struct used for state-history logging. All live simulation data (available resources, max requirements, current allocations, and remaining need) is stored in plain global arrays rather than per-process structs, keeping the design simple and directly compatible with the 2D Banker's Algorithm matrices:

```c
#include "banker.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

pthread_mutex_t sys_lock = PTHREAD_MUTEX_INITIALIZER;
bool sim_active = false;

int num_processes, num_resources;
int available[MAX_RESOURCES];
int max[MAX_PROCESSES][MAX_RESOURCES];
int allocation[MAX_PROCESSES][MAX_RESOURCES];
int need[MAX_PROCESSES][MAX_RESOURCES];
int request[MAX_PROCESSES][MAX_RESOURCES];

SystemState state_history[MAX_STATES];
int state_count = 0;

const char* mission_names[10] = {
    "Rover-A", "Ice Drill", "Hab-Build", "Drone-X", "Comms-Fix",
    "Bio-Lab", "Miner-1", "Solar-Rig", "Astro-T", "Scout-V"
};

const char* asset_names[10] = {
    "O2 Tanks", "Batteries", "Drones", "Panels", "Antennas",
    "Water", "Heaters", "Tools", "Medkits", "Rations"
};
```

6.2 Core Algorithm: banker.c

The Banker's Algorithm implementation uses global arrays and the following function signatures:

Safety Check Function: Verifies whether the current allocation state is safe.

```c
bool is_safe(char safe_seq[]) {
    int work[MAX_RESOURCES];
    bool finish[MAX_PROCESSES] = {false};
    for (int i = 0; i < num_resources; i++) work[i] = available[i];
    int safe_count = 0;
    int seq_idx = 0;
    while (safe_count < num_processes) {
        bool found = false;
        for (int p = 0; p < num_processes; p++) {
            if (!finish[p]) {
                bool can_allocate = true;
                for (int j = 0; j < num_resources; j++) {
                    if (need[p][j] > work[j]) {
                        can_allocate = false;
                        break;
                    }
                }
                if (can_allocate) {
                    for (int j = 0; j < num_resources; j++) work[j] += allocation[p][j];
                    finish[p] = true;
                    safe_seq[seq_idx++] = 'M';
                    safe_seq[seq_idx++] = '0' + p;
                    safe_seq[seq_idx++] = ' ';
                    safe_count++;
                    found = true;
                }
            }
        }
        if (!found) return false;
    }
    safe_seq[seq_idx] = '\0';
    return true;
}
```

Resource Request Function: Handles allocation requests per process.

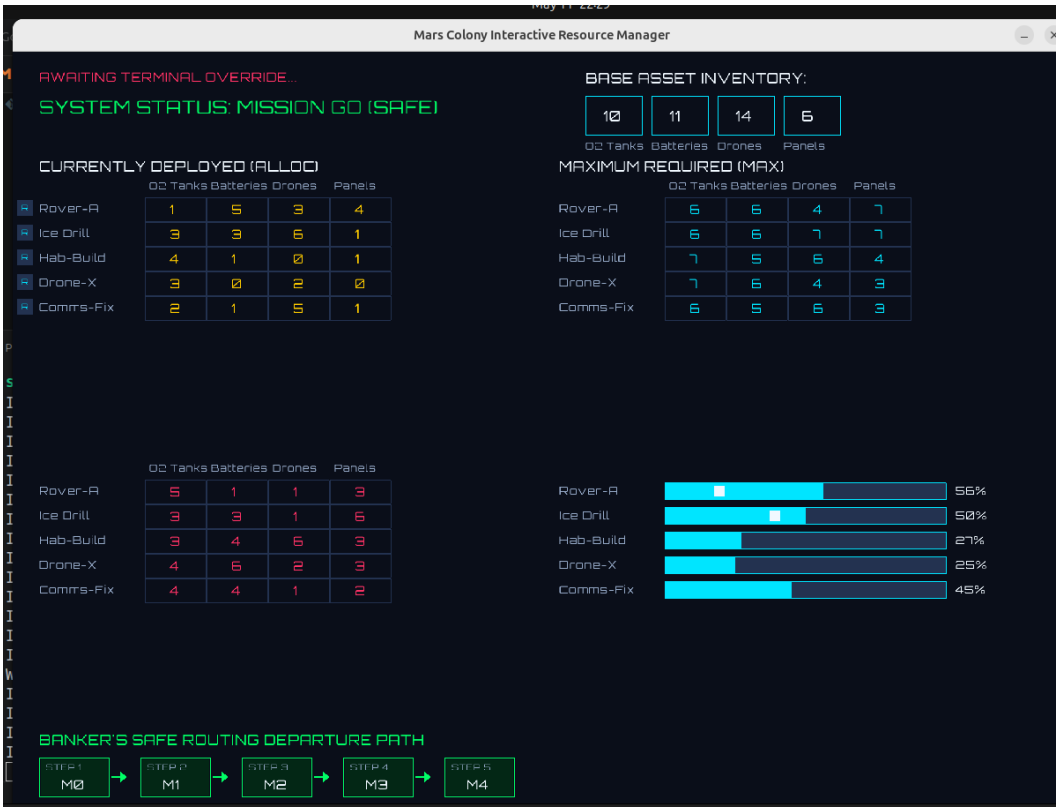
```c
bool resource_request(int process_id) {
    printf("\n=== ASSET DEPLOYMENT REQUEST FOR MISSION %d ===\n", process_id);
    int req[MAX_RESOURCES];
    for (int i = 0; i < num_resources; i++) {
        req[i] = request[process_id][i];
        printf("Request A%d: %d ", i, req[i]);
    }
    printf("\n");
    for (int i = 0; i < num_resources; i++) {
        if (req[i] > need[process_id][i]) {
            printf("[ERROR] Request exceeds maximum Mission requirements!\n");
            return false;
        }
    }
    for (int i = 0; i < num_resources; i++) {
        if (req[i] > available[i]) {
            printf("[DENIED] Insufficient assets in Base Inventory.\n");
            return false;
        }
    }
    for (int i = 0; i < num_resources; i++) {
        available[i] -= req[i];
        allocation[process_id][i] += req[i];
        need[process_id][i] -= req[i];
    }

    bool process_completes = true;
    for (int i = 0; i < num_resources; i++) {
        if (need[process_id][i] > 0) {
            process_completes = false;
            break;
        }
    }
```

6.3 Visualization: main.c + Raylib

The main visualization loop uses Raylib to:

- Render Mars colony background and UI elements
- Display process/mission status with color-coded indicators
- Show resource bars (asset levels per mission)
- Animate resource requests and allocations
- Render graphical routing pathways between processes
- Display real-time statistics and safety information



7.1 Multi-threading Architecture

Each process is implemented as a separate POSIX thread (pthread). The main thread handles graphics rendering, while process threads simulate resource requests and completions. Synchronization is achieved using:

- Mutex (sys_lock): A single pthread_mutex_t protects access to the shared global allocation arrays — all threads lock sys_lock before reading or writing simulation state.
- Polling with usleep(): Rather than blocking on condition variables, threads use usleep() to wait betweenrequest attempts, periodically retrying until resources become available.
- Atomic State Updates: The lock-tentative-allocate-check-confirm/rollback pattern inside resource_request() ensures thread-safe state transitions.

7.2 Resource Allocation Flow

1. Process thread calls resource_request(process_id)
2. System locks sys_lock (pthread_mutex_lock)
3. Banker's Algorithm safety check is executed via is_safe()
4. If safe: resources allocated and state updated; if unsafe: request denied and rolled back
5. sys_lock is released; visualization is updated
6. Process continues execution with allocated resources
7. When complete, process releases resources (available[] restored)

7.3 Safety Algorithm Implementation

The safety check algorithm in is_safe() maintains a work vector and a finish vector:

- Work Vector: Tracks tentatively available resources, initialized from the global available[] array.
- Finish Vector: Boolean array tracking which processes can complete.
- Iteratively finds processes whose need[] can be satisfied by work[]; releases their allocation back.
- If all processes finish: state is safe (returns true); otherwise unsafe (returns false).
- Time complexity: $O(n^2 \times m)$ where n = number of processes, m = number of resources.

8.1 Interactive Features

Feature	Description
Start/Stop	Begin and pause the simulation at any time
Process Configuration	Number of processes and resources configured at startup via CLI prompt (scanf), then fixed for t
Random / Interactive Init	Choose between randomized ARES-1 conditions or manually enter all resource values through t
Stress Testing	Run maximum load scenarios to test algorithm robustness under concurrent requests
Statistics Display	Real-time metrics on allocations, denials, safe states, and gridlock risks prevented
Mission Log	display_history() shows a timestamped log of all recorded system states

8.2 Visual Enhancements

The Mars Colony theme provides an engaging educational experience:

- Thematic Graphics: Colony buildings, resource icons, and Mars-themed UI
- Color Coding: Different colors indicate process/mission states (waiting, running, completed)
- Progress Bars: Visual representation of resource allocation percentages
- Graphical Routing: Lines show connections and data flow between processes
- Smooth Animations: Fluid transitions and visual feedback for actions
- Custom Font: Orbitron.ttf loaded via Raylib for the Mars Colony aesthetic

8.3 Performance Characteristics

The system is optimized for educational purposes while maintaining realism:

- Concurrent Processes: Handles up to 10 simultaneous processes (MAX_PROCESSES = 10)
- Resource Types: Manages up to 10 different resource types (MAX_RESOURCES = 10)
- State History: Logs up to 100 system snapshots (MAX_STATES = 100)
- Safety Checks: Performed in O(n2 x m) time
- Frame Rate: Maintains smooth 60 FPS rendering via Raylib
- Stress Test: Can handle rapid concurrent allocation requests under mutex protection

GitHub Repository: https://github.com/rouq-alt/Mars-colony-interactive-resource-handler

File Organization

File	Purpose
banker.h	Core data structures, global array declarations, and function prototypes
banker.c	Banker's Algorithm implementation: is_safe(), resource_request(), init_random_system(), init_int
main.c	Raylib graphics initialization and simulation loop
cli.c	Command-line interface for system configuration and thread entry point (cli_thread_function)
Makefile	Build configuration and compilation rules
resources/	Graphics assets and theme resources
Orbitron.ttf	Custom font for Mars Colony theme (loaded by Raylib at runtime)

The Mars Colony Interactive Resource Handler successfully demonstrates the Banker's Algorithm for deadlock avoidance in a visually engaging and interactive manner. The project combines fundamental operating system concepts with modern graphics programming to create an educational tool that helps students understand complex synchronization and resource allocation mechanisms.

Key Achievements

- Successfully implemented the Banker's Algorithm in C using global 2D arrays with correct is_safe(charsafe_seq[]) and resource_request(int process_id) signatures
- Created an intuitive and visually appealing user interface using Raylib
- Demonstrated safe resource allocation under concurrent access using a single mutex (sys_lock) and ausleep()-based polling model
- Provided real-time visualization and interactive controls
- Tested system stability and performance under stress conditions

Learning Outcomes

Students using this application can understand:

- How deadlock detection and avoidance algorithms work
- Multi-threading and mutex-based synchronization in practice
- Real-time graphics rendering in C using Raylib
- Resource management in operating systems
- The importance of formal verification in concurrent systems

---

Report Generated: May 11, 2026 at 16:06
Mars Colony Interactive Resource Handler — CS 3012 Operating Systems Project
Authors: Nimra Mehmood (24K-0807), Shorouq Iqbal (24K-0914)